Density-Dependent Marked Point Process Framework (DenMark) Tutorial

Mingchi Xu, Alexandra M. Schmidt** and Qihuang Zhang*

October 10, 2026

Contents

0. Prepare packages and functions **2**

1. Load single-cell resolution ST data and select hyperparameters **3**

2. ROI selection, gene filtering and gridding **4**

3. Single-gene analysis: quantify the density-expression association **7**
3.1 Fit the full and the simplified DenMark models 7
3.2 Posterior summaries 8
3.3 Model comparison and diagnostics 9
3.4 Latent field decomposition 10
3.5 Spatial correlation functions and ranges 11

4. DCG discovery: identify the density-correlated genes **12**

5. The same workflow in Python **13**

Session information **14**

This tutorial shows how to use **DenMark** to quantify the correlation between gene expression and cell density in single-cell resolution spatial transcriptomics (ST) data, and how to identify the density-correlated genes (DCGs). All analyses use the user-level functions in `DenMark-functions.R` together with the two Stan models:

- `DenMark-M2.stan`: the full DenMark model, with dependent density and expression latent fields;
- `DenMark-M1.stan`: the simplified DenMark model, with independent fields and $a_{21} = 0$, used for model comparison.

For a grid cell g with area $|g|$, cell count $Y(g)$ and summed expression $M(g)$, the full model is

$$\begin{aligned} Y(g) &\sim \text{Poisson}\{\lambda_1(g)\}, \quad \log \lambda_1(g) = \log |g| + \beta_0 + a_{11} w_1(g), \\ M(g) \mid Y(g) &\sim \text{Poisson}\{\lambda_2(g)\}, \quad \log \lambda_2(g) = \beta_1 + \log\{Y(g) + 0.05\} + a_{21} w_1(g) + a_{22} w_2(g), \end{aligned}$$

where w_1, w_2 are independent unit-variance Matérn-3/2 Gaussian processes, approximated with Hilbert-space basis functions (HSGP). With $a_{11} = \sigma_1$, $a_{21} = \sigma_2 \rho$ and $a_{22} = \sigma_2 \sqrt{1 - \rho^2}$, the parameter ρ, named `Astar[2,1]` in Stan, is the **density-expression correlation**.

We use the 10x Xenium human breast cancer dataset (In Situ Sample 1, Replicate 1). We start with two files:

- `cell_spatial.csv`: the 2D locations and cell types of the cells, one row per cell ID;

- `cell_by_gene_expression.csv`: the raw gene expression of the cells, one row per cell ID and one column per gene.

Section 1 loads the data and sets the hyperparameters. Section 2 selects the region of interest (ROI), filters the genes and grids the data. Section 3 quantifies the density-expression relationship for one gene. Section 4 identifies the DCGs, and Section 5 shows the same workflow in Python.

> **Running this file.** Put `DenMark-functions.R`, `DenMark-M1.stan` and `DenMark-M2.stan` in the folder given by the `denmark_dir` parameter, and the two CSV files in `data_dir`. Then knit, or run `rmarkdown::render("Tutorial_DenMark.Rmd")`. If the CSV files are not found, the tutorial runs on a **simulated** dataset with known correlations, so the whole document can always be reproduced. Fitted models are saved in `fit_dir` and reused on the next knit. Set `refit: true` to refit.

---

0. Prepare packages and functions

DenMark runs Stan through **cmdstanr**, which needs a working CmdStan installation. Install both once, following https://mc-stan.org/cmdstanr/:

```
# [Code Block 0.1] One-time installation
install.packages("cmdstanr",
                 repos = c("https://stan-dev.r-universe.dev", getOption("repos")))
cmdstanr::check_cmdstan_toolchain(fix = TRUE)
cmdstanr::install_cmdstan(cores = 4)

# packages used by the DenMark functions and this tutorial
install.packages(c("posterior", "loo", "ggplot2", "patchwork", "knitr"))
```

Load the packages and the DenMark functions. `rstan` and `rethinking` are no longer needed.

```
# [Code Block 0.2] Load packages and DenMark functions
library(cmdstanr)
library(posterior)
library(loo)
library(ggplot2)
library(patchwork)
library(knitr)

if (nzchar(params$cmdstan_path)) set_cmdstan_path(params$cmdstan_path)
cmdstan_version()
```

```
## [1] "2.36.0"
```

```
# DenMark functions; the Stan files are looked up in the same folder
source(file.path(params$denmark_dir, "DenMark-functions.R"))
options(denmark.stan_dir = params$denmark_dir)

# a compact table printer for this document
show_table <- function(x, digits = 3, ...) {
  kable(x, digits = digits, booktabs = TRUE, row.names = FALSE, ...)
}
```

The functions used in this tutorial are:

Step	Function	Purpose
QC	`denmark_filter_genes()`	keep informative genes
Gridding	`buildgridpp()`, `grid_pts_centroid()`	cell counts Y, expression M, grid geometry
Fitting	`DenMark()`	fit M2 (full) or M1 (simplified) to one gene
Summaries	`denmark_summary()`, `denmark_draws()`, `denmark_compare()`	posterior summary, ρ draws, WAIC
Interpretation	`denmark_fields()`, `denmark_decay()`	latent field decomposition, spatial correlation ranges
DCG discovery	`denmark_scan()`	fit many genes, classify PCG / NCG
Plots	`plot_denmark_grid()`, `plot_denmark_fields()`, `plot_denmark_decay()`, `plot_denmark_scan()`	figures

1. Load single-cell resolution ST data and select hyperparameters

```
# [Code Block 1.1] Locate the data
f_gene    <- file.path(params$data_dir, "cell_by_gene_expression.csv")
f_spatial <- file.path(params$data_dir, "cell_spatial.csv")
use_xenium <- file.exists(f_gene) && file.exists(f_spatial)
use_xenium
```

```
## [1] FALSE
```

```
# [Code Block 1.2b] No Xenium files found: simulate data with known rho.
# Three genes: geneA with rho = 0.6, geneB with rho = 0, geneC with rho = -0.6
sim <- denmark_simulate(rho = c(geneA = 0.6, geneB = 0, geneC = -0.6),
                        width = 6, height = 4, n_cells = 5000, seed = 1)
cell_spatial <- data.frame(X = seq_len(nrow(sim)) - 1, x = sim$x, y = sim$y,
                           celltype = params$celltype)
cell_gene <- data.frame(cell_id = cell_spatial$X, sim[, c("geneA", "geneB", "geneC")])
cell_gene$lowGene <- rpois(nrow(sim), 0.01)  # rarely detected: removed by QC below
head(cell_spatial)
```

```
##   X        x        y       celltype
## 1 0 5.733302 2.097423 Invasive_Tumor
## 2 1 5.376683 1.037494 Invasive_Tumor
## 3 2 2.375250 2.305503 Invasive_Tumor
## 4 3 0.522338 2.285421 Invasive_Tumor
## 5 4 5.299141 2.434258 Invasive_Tumor
## 6 5 2.105266 3.907017 Invasive_Tumor
```

```
gene_choice <- "geneA"
scan_genes  <- c("geneA", "geneB", "geneC")
unit_label  <- "mm (simulated)"
```

The hyperparameters control the approximation precision:

- `grid_size`: number of grid cells per axis, given as a single number or as `c(nx, ny)`;
- `basis`: number of HSGP basis functions per axis. More basis functions approximate shorter length-scales better, at a higher computational cost;
- `boundfactor`: HSGP boundary factor $c \geq 1$, which removes boundary effects; $c = 1.5$ is a good default.

```r
# [Code Block 1.3] Select hyperparameters
grid_size       <- params$grid_size
basis_choice    <- params$basis
boundary_choice <- params$boundfactor

# sampler settings used for every fit in this tutorial
mcmc <- list(chains = params$chains, iter_warmup = params$iter_warmup,
             iter_sampling = params$iter_sampling, adapt_delta = 0.95,
             max_treedepth = 10, seed = 2024)
c(grid_size = grid_size, basis = basis_choice, boundfactor = boundary_choice)
```

```
##   grid_size       basis boundfactor
##        10.0        12.0         1.5
```

2. ROI selection, gene filtering and gridding

The ROI is chosen to suit the research question. Here we run DenMark on all cells of one cell type across the whole tissue, which is a large-range analysis. A smaller window can be analyzed the same way by subsetting the cells, or by passing `bbox = c(xmin, xmax, ymin, ymax)` to the functions.

```r
# [Code Block 2.1] Extract the cell type of interest
sel <- which(cell_spatial$celltype == params$celltype)
cells_roi <- cell_spatial[sel, ]
gene_roi  <- cell_gene[sel, -1, drop = FALSE]  # drop the cell ID column
cat(length(sel), "cells of type", params$celltype, "\n")
```

```
## 5000 cells of type Invasive_Tumor
```

```r
ggplot(cells_roi, aes(x, y)) +
  geom_point(size = 0.15, alpha = 0.6) +
  coord_equal() +
  labs(x = paste0("x (", unit_label, ")"), y = paste0("y (", unit_label, ")")) +
  theme_classic()
```

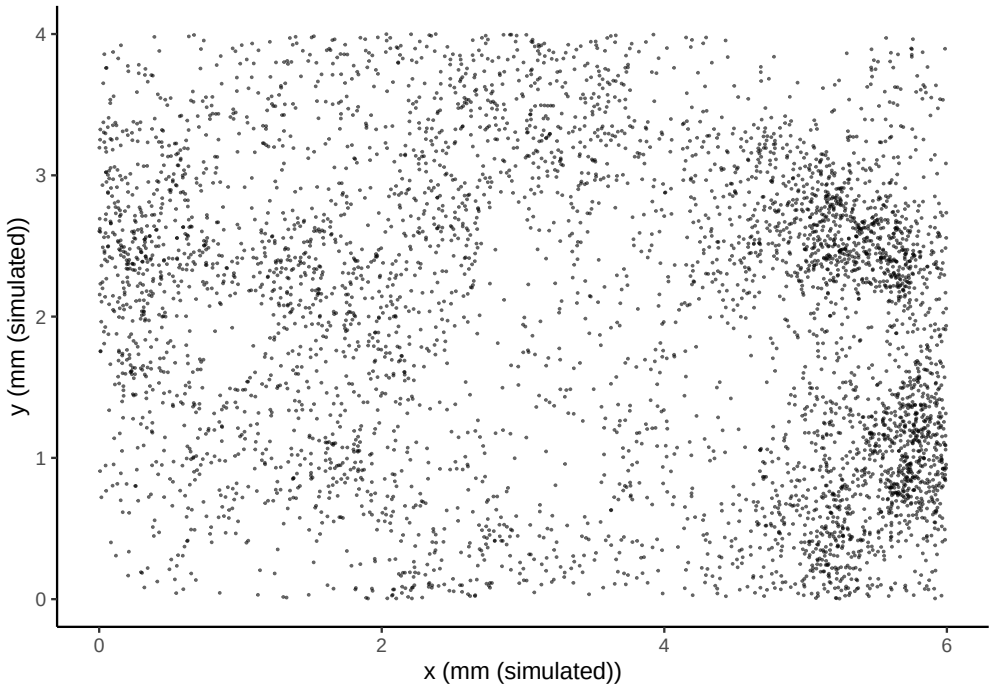


Figure 1: Cells of the selected cell type.

```
# Short-range analysis: e.g. a pathologist-selected window [0.5, 1.5] x [2, 3]
# keep <- with(cell_spatial, x >= 0.5 & x <= 1.5 & y >= 2 & y <= 3)
```

Quality control. Genes in the lowest 5% of variance across cells, or detected in fewer than 5% of the cells, are removed.

```
# [Code Block 2.2] Filter uninformative genes
genes_keep <- denmark_filter_genes(gene_roi, var_quantile = 0.05, min_nonzero = 0.05)
length(genes_keep)
```

```
## [1] 3
```

```
head(genes_keep, 10)
```

```
## [1] "geneA" "geneB" "geneC"
```

```
# combined data set for DenMark: one row per cell, columns x, y, genes
final_df <- data.frame(x = cells_roi$x, y = cells_roi$y,
                       gene_roi[, genes_keep, drop = FALSE], check.names = FALSE)
final_df[1:3, 1:min(6, ncol(final_df))]
```

```
##          x        y geneA geneB geneC
## 1 5.733302 2.097423     0     0     0
## 2 5.376683 1.037494     1     9     5
## 3 2.375250 2.305503     3     0     4
```

We analyze one gene in detail. The input to DenMark is a data frame with the columns `x`, `y` and the gene's counts.

```
# [Code Block 2.3] Select the gene of interest
stopifnot(gene_choice %in% names(final_df))
df_gene <- final_df[, c("x", "y", gene_choice)]
```

```
ggplot(df_gene, aes(x, y, color = .data[[gene_choice]])) +
  geom_point(size = 0.3) +
  scale_color_gradient(low = "gray92", high = "red", trans = "sqrt") +
  coord_equal() +
  labs(x = paste0("x (", unit_label, ")"), y = paste0("y (", unit_label, ")"),
       color = gene_choice) +
  theme_classic()
```

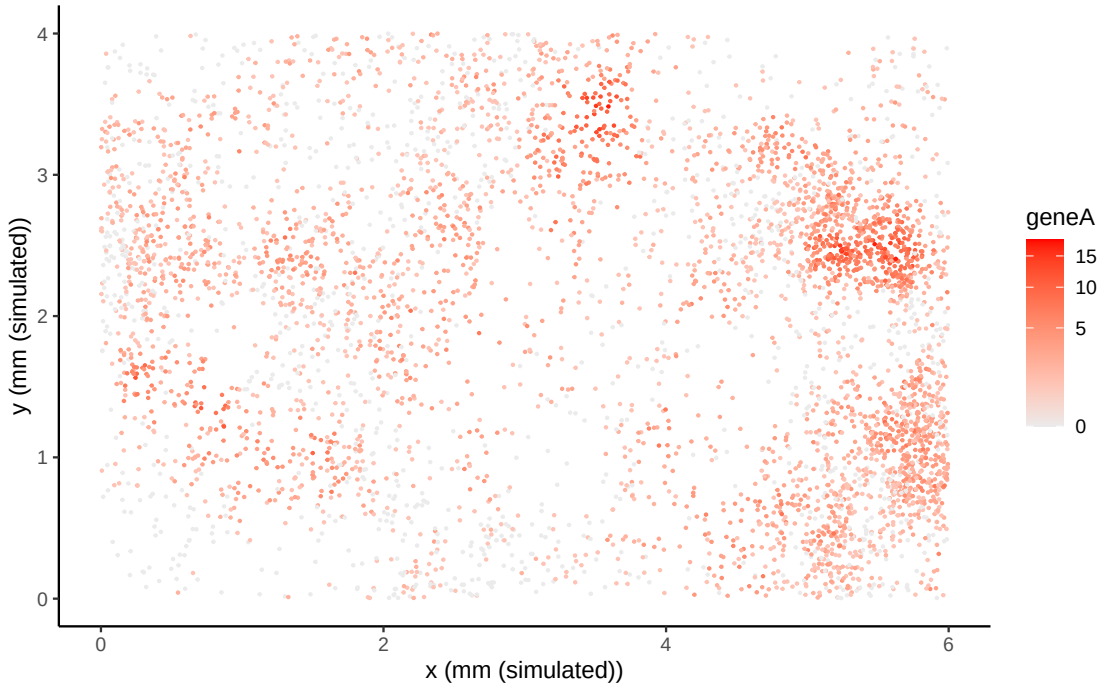


Figure 2: Cell-level expression of the selected gene.

Gridding. `buildgridpp()` assigns every cell to exactly one grid cell and returns the counts $Y(g)$ and the summed expression $M(g)$, both as vectors and as matrices. `grid_pts_centroid()` returns the grid geometry. DenMark centers the ROI at the origin, so that it becomes $[-L_1, L_1] \times [-L_2, L_2]$, and uses the grid centroids as the HSGP coordinates. This is done inside `DenMark()`, so you no longer need to compute L_1 and L_2 yourself.

```
# [Code Block 2.4] Grid the data
grid_res <- buildgridpp(grid_size = grid_size, dataset = df_gene)
names(grid_res)
```

```
## [1] "loc_mark_mat" "Ymat"         "Mmat"         "grid"         "bbox"
## [6] "step"         "grid_area"    "grid_size"    "gene"
```

```
# grid_res[[1]]: N x 2 matrix of cell counts Y and expression M
# grid_res[[2]]: matrix of cell counts;  grid_res[[3]]: matrix of expression
head(grid_res$grid)
```

```
##   grid_index i j         x         y  Y  M
## 1          1 1 1 0.3015223 0.2036139 17  2
## 2          2 2 1 0.9009915 0.2036139  8  1
## 3          3 3 1 1.5004608 0.2036139 10  3
```

```
## 4          4 4 1 2.0999300 0.2036139 50 32
## 5          5 5 1 2.6993992 0.2036139 39  9
## 6          6 6 1 3.2988685 0.2036139 30 14
```

```
geom <- grid_pts_centroid(grid_size = grid_size, dataset = df_gene)
c(grid_area = geom$grid_area, n_grids = geom$N)
```

```
##  grid_area    n_grids
##   0.239531 100.000000
```

```
# the Stan input that DenMark() builds internally, useful for checks
prep <- denmark_prepare(df_gene, gene = gene_choice, grid_size = grid_size,
                        basis = basis_choice, boundfactor = boundary_choice)
c(L1 = prep$info$L1, L2 = prep$info$L2, HSGP_basis_total = prep$stan_data$mstar)
```

```
##               L1               L2 HSGP_basis_total
##         2.997346         1.997859       144.000000
```

```
# [Code Block 2.5] Visualize the gridded data
plot_denmark_grid(grid_res, xlab = unit_label, ylab = unit_label)
```

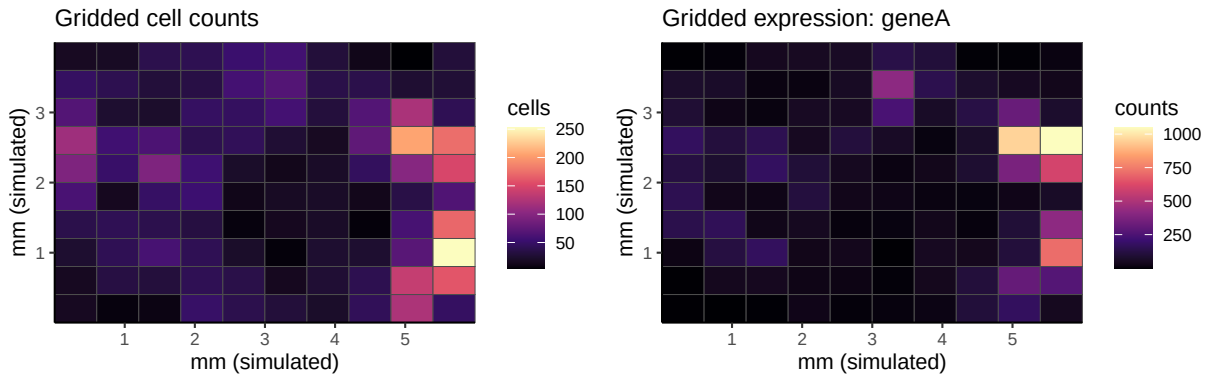


Figure 3: Gridded cell counts and gene expression.

3. Single-gene analysis: quantify the density-expression association

3.1 Fit the full and the simplified DenMark models

`DenMark()` grids the data, builds the HSGP inputs, compiles the Stan model once per session, sets valid initial values and samples. `model = "M2"` fits the full model and `model = "M1"` the simplified model. The fits are saved with `denmark_save()` so they can be reloaded without refitting.

```
# [Code Block 3.1] Run the full and simplified DenMark versions
fit_or_load <- function(file, ...) {
  if (!params$refit && file.exists(file)) return(denmark_load(file))
  fit <- do.call(DenMark, c(list(...), mcmc))
  denmark_save(fit, file)
  fit
```

```
}
tag <- paste(gene_choice, grid_size, basis_choice, params$iter_sampling, sep = "_")

denmark_full <- fit_or_load(file.path(params$fit_dir, paste0("M2_", tag, ".rds")),
                            dataset = df_gene, gene = gene_choice, model = "M2",
                            grid_size = grid_size, basis = basis_choice,
                            boundfactor = boundary_choice, quiet = TRUE)

denmark_simple <- fit_or_load(file.path(params$fit_dir, paste0("M1_", tag, ".rds")),
                              dataset = df_gene, gene = gene_choice, model = "M1",
                              grid_size = grid_size, basis = basis_choice,
                              boundfactor = boundary_choice, quiet = TRUE)
denmark_full
```

```
## DenMark fit  |  model M2  |  gene geneA
##   grid 10 x 10 (100 grids, 5000 cells)  |  basis 12  |  boundfactor 1.50
##   rho (density-expression correlation): 0.625  [95% CrI 0.431, 0.773]
##   WAIC: 1478.93 (SE 21.73)
##   max R-hat 1.005 | min ESS bulk 572 | divergences 0 | time 1.13 min
##
##    variable   mean     sd   q2.5    q50 q97.5 rhat ess_bulk
##       beta0 4.9577 0.1986  4.546 4.9623 5.345    1     1650
##       beta1 0.0196 0.2588 -0.534 0.0385 0.468    1     1195
##      ell[1] 0.5250 0.1105  0.342 0.5130 0.758    1      697
##      ell[2] 0.5101 0.1399  0.284 0.4975 0.820    1      612
##  Astar[2,1] 0.6247 0.0874  0.431 0.6341 0.773    1      638
##    sigma[1] 0.8601 0.0966  0.708 0.8506 1.070    1     1028
##    sigma[2] 1.0684 0.1393  0.851 1.0510 1.383    1      572
```

3.2 Posterior summaries

For the full model: β_0, β_1 are the intercepts of the cell density and gene expression intensities, `ell[1]`, `ell[2]` the length-scales of the two latent fields, `Astar[2,1]` $= \rho$ the density-expression correlation, and `sigma[1]`, `sigma[2]` the standard deviations of the two fields. R-hat close to 1 and large effective sample sizes (ESS) indicate convergence.

```
# [Code Block 3.2] Posterior summaries
cols <- c("variable", "mean", "sd", "q2.5", "q50", "q97.5", "rhat",
          "ess_bulk", "ess_tail")
show_table(denmark_summary(denmark_full)[, cols], caption = "Full DenMark (M2).")
```

Table 2: Full DenMark (M2).

variable	mean	sd	q2.5	q50	q97.5	rhat	ess_bulk	ess_tail
beta0	4.958	0.199	4.546	4.962	5.345	1.002	1650.148	1036.450
beta1	0.020	0.259	-0.534	0.038	0.468	1.005	1195.266	838.888
ell[1]	0.525	0.111	0.342	0.513	0.758	1.001	696.820	1077.837
ell[2]	0.510	0.140	0.284	0.497	0.820	1.000	612.499	1018.604
Astar[2,1]	0.625	0.087	0.431	0.634	0.773	1.003	638.357	920.326
sigma[1]	0.860	0.097	0.708	0.851	1.070	1.002	1028.065	1116.596
sigma[2]	1.068	0.139	0.851	1.051	1.383	1.002	571.935	965.162

```
show_table(denmark_summary(denmark_simple)[, cols],
           caption = "Simplified DenMark (M1): a11 and a22 are the field SDs.")
```

Table 3: Simplified DenMark (M1): a11 and a22 are the field SDs.

variable	mean	sd	q2.5	q50	q97.5	rhat	ess_bulk	ess_tail
beta0	4.979	0.210	4.563	4.979	5.392	1.001	2215.426	1210.728
beta1	0.163	0.155	-0.159	0.174	0.448	1.005	2135.360	1168.271
ell[1]	0.614	0.120	0.411	0.599	0.872	1.003	802.051	1105.423
ell[2]	0.345	0.075	0.219	0.337	0.508	1.001	1375.020	1336.642
a11	0.829	0.101	0.666	0.821	1.064	1.003	880.997	1005.428
a22	0.954	0.098	0.782	0.949	1.160	1.005	631.738	1196.569

```
# [Code Block 3.3] Posterior of rho
draws_full <- denmark_draws(denmark_full)   # rho, a11, a21, a22, ell1, ell2, ...
rho_ci <- quantile(draws_full$rho, c(0.025, 0.975))

ggplot(draws_full, aes(rho)) +
  geom_density(fill = "gray80") +
  geom_vline(xintercept = 0, linetype = 2) +
  geom_vline(xintercept = rho_ci, color = "firebrick", linetype = 3) +
  xlim(-1, 1) +
  labs(x = expression(rho), y = "Posterior density") +
  theme_classic()
```

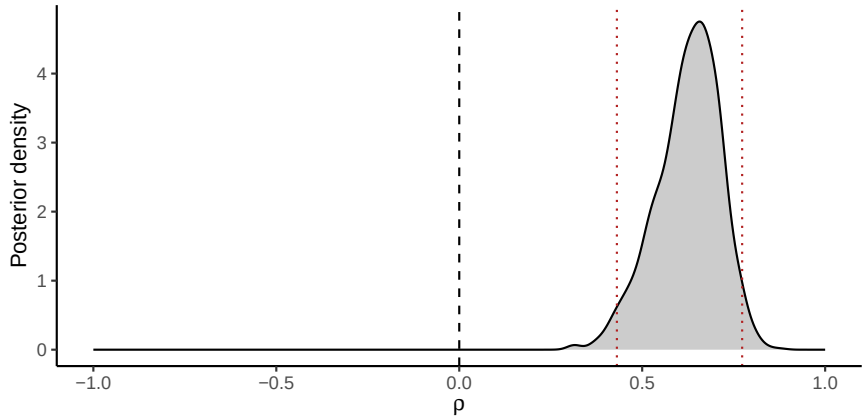


Figure 4: Posterior distribution of the density-expression correlation.

The posterior mean of ρ for geneA is **0.625** with 95% credible interval (0.431, 0.773).

3.3 Model comparison and diagnostics

```
# [Code Block 3.4] WAIC comparison: lower WAIC is better
cmp <- denmark_compare(full_M2 = denmark_full, simplified_M1 = denmark_simple)
show_table(cmp[, c("model", "waic", "waic_se", "p_waic", "elpd_diff", "se_diff")],
           digits = 2)
```

model	waic	waic_se	p_waic	elpd_diff	se_diff
full_M2	1478.93	21.73	101.39	0.00	0.00
simplified_M1	1497.32	20.82	108.54	-9.19	3.91

```
# sampler diagnostics and running time
diag_row <- function(f) with(f$diagnostics, data.frame(
  divergences = num_divergent, treedepth_hits = num_max_treedepth,
  max_rhat = max_rhat, min_ess_bulk = min_ess_bulk, minutes = f$time))
show_table(cbind(model = c("M2", "M1"),
                 rbind(diag_row(denmark_full), diag_row(denmark_simple))), digits = 3)
```

model	divergences	treedepth_hits	max_rhat	min_ess_bulk	minutes
M2	0	0	1.005	571.935	1.134
M1	0	0	1.005	631.738	0.733

`elpd_diff` is the difference in expected log predictive density relative to the best model, and `se_diff` is its standard error. A difference of several standard errors in favor of M2 supports a density-expression dependence.

The HSGP approximation can be checked against the rule of thumb of Riutort-Mayol et al. (2023). If the posterior length-scale is shorter than `ell_min`, increase `basis`. If `boundary_ok` is `FALSE`, increase `boundfactor`. The rule is conservative, so treat it as guidance.

```
# [Code Block 3.5] HSGP adequacy check
show_table(denmark_check_hsgp(denmark_full, warn = FALSE))
```

field	axis	ell_median	ell_min	c_needed	basis_ok	boundary_ok
density	x	0.513	1.281	1.2	FALSE	TRUE
expression	x	0.497	1.281	1.2	FALSE	TRUE
density	y	0.513	0.854	1.2	FALSE	TRUE
expression	y	0.497	0.854	1.2	FALSE	TRUE

3.4 Latent field decomposition

`denmark_fields()` returns, for every grid cell, the posterior means of the log cell density $\beta_0 + a_{11}w_1$ and the log expression per cell $\beta_1 + a_{21}w_1 + a_{22}w_2$, decomposed into the density field $a_{11}w_1$, the density-shared expression component $a_{21}w_1$, and the expression-specific component $a_{22}w_2$.

```
# [Code Block 3.6] Latent field decomposition
fields_full <- denmark_fields(denmark_full)
head(fields_full[, c("grid_index", "Y", "M", "eta1", "a11w1", "eta2", "a21w1", "a22w2")], 3)
```

```
##   grid_index  Y M     eta1      a11w1       eta2      a21w1      a22w2
## 1          1 17 2 4.210522 -0.7472156 -1.6262252 -0.5839896 -1.0618842
## 2          2  8 1 3.695791 -1.2619474 -1.4650524 -0.9871270 -0.4975740
## 3          3 10 3 3.983784 -0.9739540 -0.8808672 -0.7618486 -0.1386672
```

```
plot_denmark_fields(denmark_full, fields = fields_full, xlab = unit_label, ylab = unit_label)
```

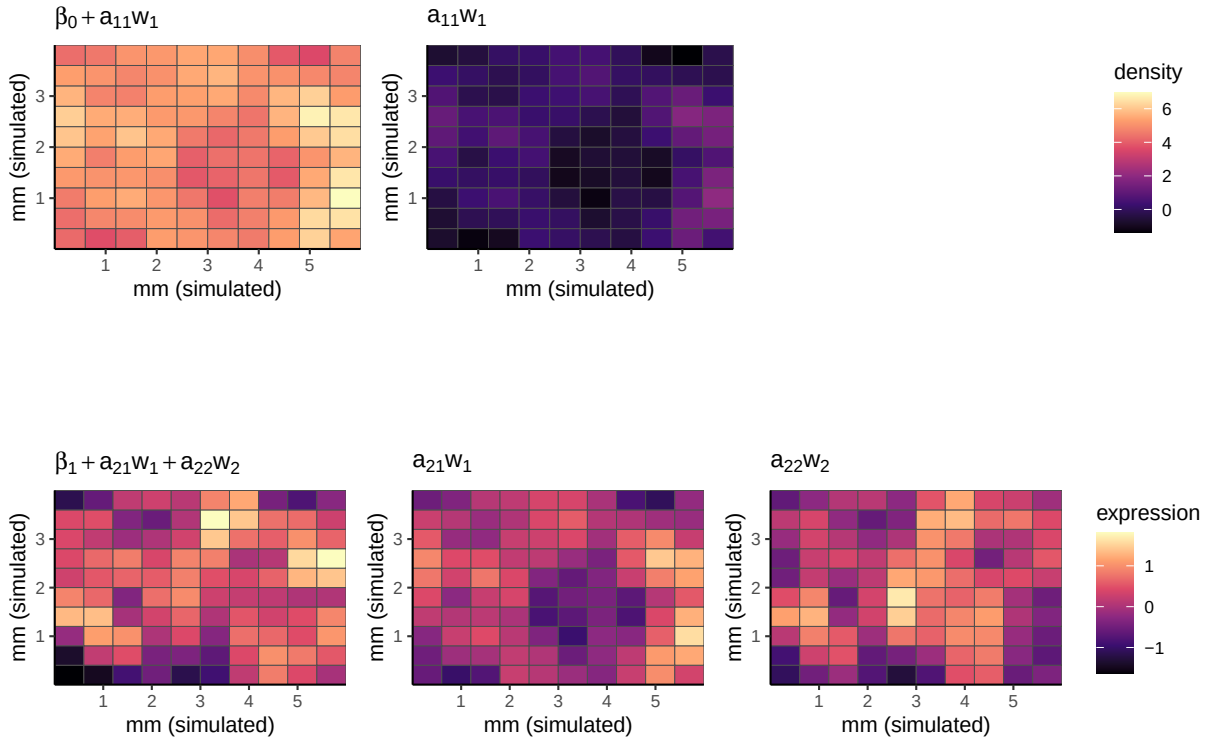


Figure 5: Latent field decomposition (posterior means). Top: cell density; bottom: gene expression.

3.5 Spatial correlation functions and ranges

The density field decays as $r_1(d) = \mathcal{M}(d; \ell_1)$. The expression field decays as $r_2(d) = \{a_{21}^2 \mathcal{M}(d; \ell_1) + a_{22}^2 \mathcal{M}(d; \ell_2)\}/(a_{21}^2 + a_{22}^2)$, where $\mathcal{M}$ is the Matérn-3/2 correlation. The practical range is the distance at which the correlation drops to 0.05. It is computed for every posterior draw.

```
# [Code Block 3.7] Spatial correlation decay
decay_full <- denmark_decay(denmark_full, threshold = 0.05)
plot_denmark_decay(denmark_full, decay = decay_full,
                   xlab = paste0("Distance (", unit_label, ")"))
```

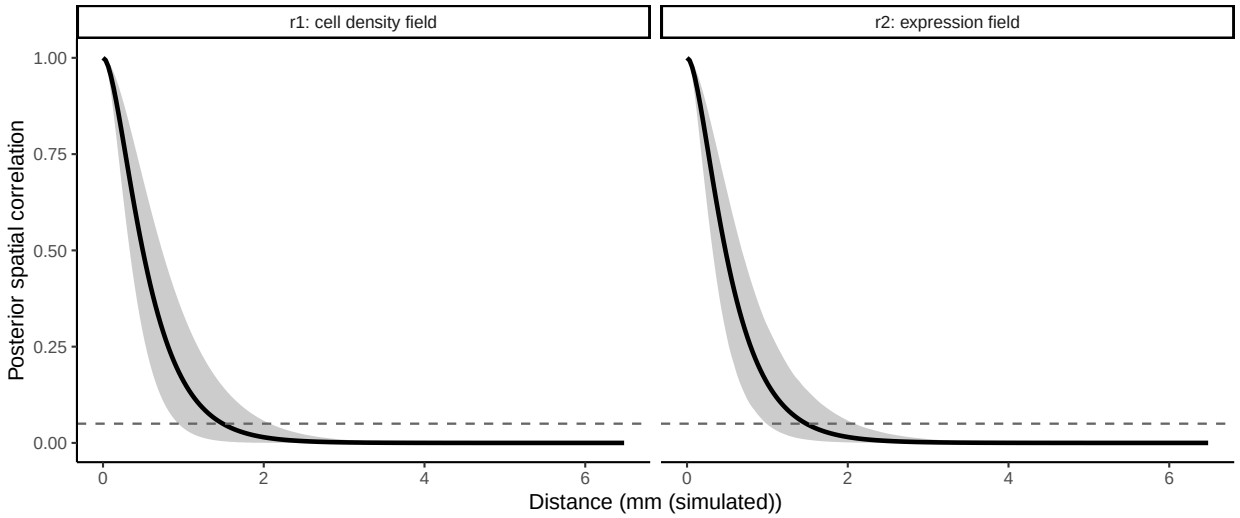


Figure 6: Posterior spatial correlation functions (mean and 95% band).

```
show_table(decay_full$ranges, caption = "Practical ranges (distance where correlation = 0.05).")
```

Table 7: Practical ranges (distance where correlation = 0.05).

field	mean	median	lower	upper
density	1.438	1.405	0.937	2.077
expression	1.445	1.413	0.987	2.084

The estimated range is 1.44 mm (simulated) for cell density and 1.45 mm (simulated) for the expression of geneA.

4. DCG discovery: identify the density-correlated genes

`denmark_scan()` fits the full model to each candidate gene and classifies it from the 95% credible interval of ρ:

- **PCG**, a positively density-correlated gene: the interval lies above 0;
- **NCG**, a negatively density-correlated gene: the interval lies below 0;
- **not DCG**: the interval contains 0.

Here we scan a few genes. For the whole panel, pass all filtered genes, set `save_dir` so that each gene is saved as it finishes and an interrupted scan resumes where it stopped, and consider running the genes in parallel on a cluster. `fit_simple = TRUE` also fits M1 and reports the WAIC difference per gene.

```
# [Code Block 4.1] Scan candidate genes
scan_genes <- intersect(scan_genes, names(final_df))
dcg <- do.call(denmark_scan,
               c(list(data = final_df, genes = scan_genes,
                      grid_size = grid_size, basis = basis_choice,
                      boundfactor = boundary_choice, verbose = FALSE,
                      save_dir = file.path(params$fit_dir, paste0("scan_", tag))),
                 mcmc))
show_table(setNames(dcg[, c("gene", "rho_mean", "rho_lower", "rho_upper",
```

```
                    "class", "max_rhat")],
          c("gene", "rho", "2.5%", "97.5%", "class", "R-hat")))
```

gene	rho	2.5%	97.5%	class	R-hat
geneA	0.625	0.431	0.773	PCG	1.005
geneB	0.248	-0.029	0.516	not DCG	1.004
geneC	-0.602	-0.764	-0.404	NCG	1.004

```
# [Code Block 4.2] Forest plot of rho
plot_denmark_scan(dcg)
```

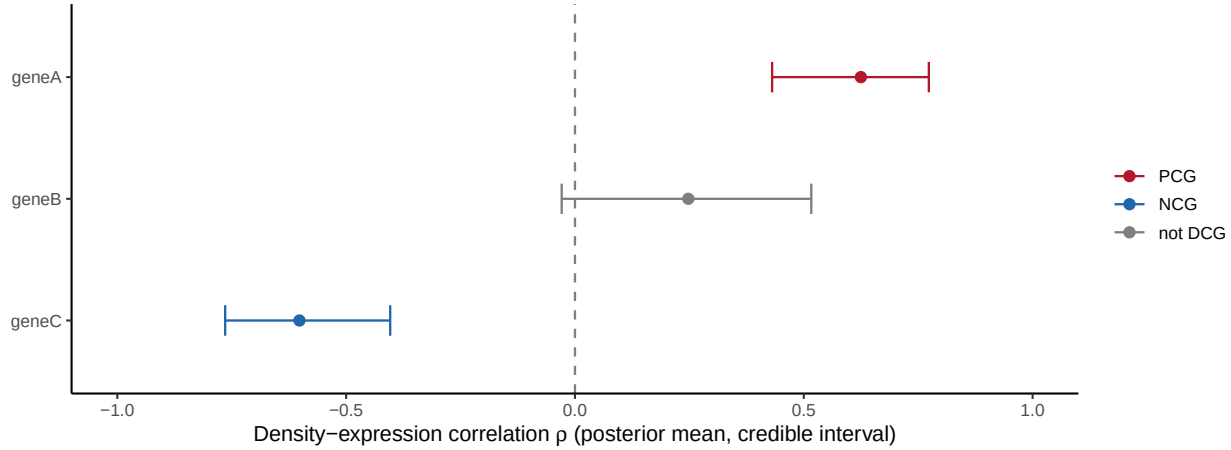


Figure 7: Density-expression correlation across genes (posterior mean and 95% credible interval).

```
# [Code Block 4.3] Full-panel scan
dcg_all <- denmark_scan(final_df, genes = genes_keep, grid_size = grid_size,
                        basis = basis_choice, boundfactor = boundary_choice,
                        save_dir = "denmark_scan_all")
table(dcg_all$class)
subset(dcg_all, class == "PCG")$gene
subset(dcg_all, class == "NCG")$gene
```

5. The same workflow in Python

`denmark.py` provides the same functions for Python, using **cmdstanpy** and the same Stan files. Given the same data, it builds identical Stan inputs and returns the same summaries.

```
# [Code Block 5.1] Python
# install once: pip install cmdstanpy pandas scipy matplotlib
#               python -m cmdstanpy.install_cmdstan
import sys; sys.path.insert(0, "DenMark")
import pandas as pd
import denmark as dm

cells = pd.read_csv("cell_spatial.csv")
genes = pd.read_csv("cell_by_gene_expression.csv")
```

```
cells[["x", "y"]] /= 1000                          # micrometers -> mm
sel = cells["celltype"] == "Invasive_Tumor"
expr = genes.loc[sel].iloc[:, 1:]
keep = dm.filter_genes(expr)                       # QC filter
df = pd.concat([cells.loc[sel, ["x", "y"]], expr[keep]], axis=1)

full   = dm.DenMark(df[["x", "y", "CDH1"]], gene="CDH1", model="M2",
                    grid_size=10, basis=25, boundfactor=1.5)
simple = dm.DenMark(df[["x", "y", "CDH1"]], gene="CDH1", model="M1",
                    grid_size=10, basis=25, boundfactor=1.5)
print(full)                                        # rho, WAIC, diagnostics
print(dm.compare_models(full_M2=full, simplified_M1=simple))
fields = full.fields(); decay = full.decay()
dm.plot_fields(full); dm.plot_decay(full)

dcg = dm.scan_genes(df, genes=["CDH1", "ERBB2", "EPCAM", "KRT8"],
                    grid_size=10, basis=25, save_dir="scan_py")
dm.plot_scan(dcg)
```

Session information

```
sessionInfo()
```

```
## R version 4.3.3 (2024-02-29)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
##
## locale:
## [1] C
##
## time zone: Etc/UTC
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] knitr_1.45      patchwork_1.2.0 ggplot2_3.4.4   loo_2.6.0       posterior_1.5.0
## [6] cmdstanr_0.9.0
##
## loaded via a namespace (and not attached):
##  [1] tensorA_0.36.2.1     utf8_1.2.4           generics_0.1.3       lattice_0.22-5
##  [5] digest_0.6.34        magrittr_2.0.3       evaluate_0.23        grid_4.3.3
##  [9] fastmap_1.1.1        jsonlite_1.8.8       Matrix_1.6-5         processx_3.8.3
## [13] backports_1.4.1      ps_1.7.6             fansi_1.0.5          viridisLite_0.4.2
## [17] scales_1.3.0         abind_1.4-5          cli_3.6.2            rlang_1.1.3
## [21] munsell_0.5.0        withr_2.5.0          yaml_2.3.8           tools_4.3.3
## [25] parallel_4.3.3       checkmate_2.3.1      dplyr_1.1.4          colorspace_2.1-0
```

```
## [29] reticulate_1.35.0    png_0.1-8           vctrs_0.6.5         R6_2.5.1
## [33] matrixStats_1.2.0    lifecycle_1.0.4     pkgconfig_2.0.3     pillar_1.9.0
## [37] gtable_0.3.4         Rcpp_1.0.12         glue_1.7.0          data.table_1.17.0
## [41] xfun_0.41            tibble_3.2.1        tidyselect_1.2.0    farver_2.1.1
## [45] htmltools_0.5.7      rmarkdown_2.25      labeling_0.4.3      compiler_4.3.3
## [49] distributional_0.4.0
```